

Testing Document and Specification

Test Reports

Team 6
CS 451R, Spring 2026

Biddinger, Jacob
Zhang, Kevin

Duong, Brian

Lei, Deming

Introduction

This document outlines the outcome of completed system tests for the Budget Bridge web application. Incidents, Defects and Changes that need to be made will be presented here formally. Although the ideas expressed here are separate entities, they will be combined into this one document.

Incidents

This section defines the incidents discovered while performing various tests on the system. This section will expand as more incidents are found. For each incident, the initial Incident ID (excluding Letter) determines the test case that it ties to. For example, Incident ID 5.4.6.4-A is an incident stemming from test case 5.4.6.4.

Incident ID	AUTH-1.1-A
Description	Registration form accepts duplicate usernames without displaying a validation error to the user, even though the backend correctly rejects the request with a 409 status code.
Originator	QA Tester
Discover Date	April 10, 2026
Severity	High
Steps Required to Produce Incident	1. Navigate to the registration page. 2. Enter a username that already exists in the system. 3. Fill in a valid email and matching passwords. 4. Click the "Register" button. 5. Observe that no error message is shown to the user despite the registration failing.
Responder	Kevin Zhang
Current Status	Closed
Cause	The frontend was not reading the error response body from the backend and therefore did not surface the error message to the user.
Resolution	Updated the frontend registration handler to parse and display the error message returned by the API.
Addressed Date	April 12, 2025
Creation Phase	Implementation
Detection Phase	Testing
Correction Time	2 Hours

Incident ID	AUTH-2.1-A
Description	Password reset link does not expire correctly. Users could reuse the same reset link multiple times within the token validity window, allowing repeated password changes from a single reset request.
Originator	QA Tester
Discover Date	April 18, 2025
Severity	High
Steps Required to Produce Incident	1. Request a password reset for an existing account 2. Click the reset link received by email 3. Enter and confirm a new valid password 4. Navigate back to the same reset link URL 5. Observe that the form accepts a second password change
Responder	Kevin Zhang
Current Status	Closed
Cause	The password reset token was not deleted from the database upon successful use, allowing the link to remain valid for its full expiration window.
Resolution	Modified the resetPassword service method to delete the token from the PasswordResetToken repository immediately after a successful reset.
Addressed Date	April 19,2026
Creation Phase	Implementation
Detection Phase	Testing
Correction Time	30 minutes

Automated Tests

This section documents the automated unit tests that have been implemented for the system. Tests are organized by component and cover backend service logic, controller endpoint behavior, and frontend component rendering. All automated tests are executed as part of the development workflow.

Backend Unit Tests - UserService

The following tests are implemented in UserServiceTest.java using JUnit 5 and Mockito. They verify the registration, login, email verification, and password reset flows of the UserService class.

Test Name	Description	Result
register_passwordMismatch_returnsFalse	Verifies that registration fails when the password and confirm password fields do not match.	Pass
register_passwordTooShort_returnsFalse	Verifies that registration fails when the password is fewer than 8 characters.	Pass
register_passwordNoNumber_returnsFalse	Verifies that registration fails when the password contains no numeric character.	Pass
register_passwordNoSpecialChar_returnsFalse	Verifies that registration fails when the password contains no special character.	Pass
register_usernameTaken_returnsFalse	Verifies that registration fails and returns the correct message when the username already exists.	Pass
register_emailTaken_returnsFalse	Verifies that registration fails and returns the correct message when the email is already registered.	Pass
register_success_returnsTrue	Verifies that a valid registration saves the user and creates a verification token.	Pass
login_userNotFound_returnsFalse	Verifies that login fails with invalid credentials when the	Pass

	username or email does not exist.	
login_wrongPassword_returnsFalse	Verifies that login fails when the correct username is given but the password is incorrect.	Pass
login_notVerified_returnsFalse	Verifies that login is blocked for accounts that have not completed email verification.	Pass
login_success_returnsTrue	Verifies that login succeeds and returns the username for valid, verified credentials.	Pass
login_byEmail_success_returnsTrue	Verifies that login succeeds when the user authenticates using their email address instead of username.	Pass
verifyEmail_invalidToken_returnsFalse	Verifies that email verification fails when the token does not exist in the database.	Pass
verifyEmail_expiredToken_returnsFalse	Verifies that an expired verification token is rejected and deleted from the database.	Pass
verifyEmail_success_returnsTrue	Verifies that a valid token marks the user as verified and is deleted after use.	Pass
resetPassword_passwordMismatch_returnsFalse	Verifies that password reset fails when the new password and	Pass

	confirmation do not match.	
resetPassword_invalidToken_returnsFalse	Verifies that password reset fails when the reset token does not exist.	Pass
resetPassword_expiredToken_returnsFalse	Verifies that an expired reset token is rejected and removed from the database.	Pass
resetPassword_success_returnsTrue	Verifies that a valid reset token updates the user password and deletes the token upon success.	Pass

Backend Unit Tests - SpendingController

The following tests are implemented in SpendingControllerTest.java using Spring Boot's @WebMvcTest framework with Mockito. They verify HTTP endpoint behavior for the expense management API, including authentication checks and CRUD operations.

Test Name	Description	Result
getExpenses_noSession_returns401	Verifies that fetching expenses without an active session returns HTTP 401 Unauthorized.	Pass
getExpenses_authenticated_returnsList	Verifies that an authenticated user receives their expense list with correct category, amount, and date fields.	Pass
getExpenses_emptyList_returnsEmptyArray	Verifies that an authenticated user with no recorded expenses receives an empty JSON array.	Pass

addExpense_noSession_returns401	Verifies that posting a new expense without a session returns HTTP 401 Unauthorized.	Pass
addExpense_missingFields_returns400	Verifies that a POST request missing required fields returns HTTP 400 Bad Request.	Pass
addExpense_valid_returnsCreatedExpense	Verifies that a valid expense submission returns the saved expense with a generated ID.	Pass
addExpense_withDescription_persistsDescription	Verifies that an optional description field is saved and returned correctly in the response.	Pass
deleteExpense_noSession_returns401	Verifies that attempting to delete an expense without a session returns HTTP 401 Unauthorized.	Pass
deleteExpense_notFound_returns404	Verifies that attempting to delete an expense that does not belong to the user returns HTTP 404 Not Found.	Pass
deleteExpense_valid_returns200AndDeletes	Verifies that a valid delete request returns HTTP 200 and calls the repository delete method.	Pass

Frontend Component Tests - Cypress

The following tests are implemented using Cypress Component Testing and Cypress unit spec files. They verify frontend rendering, style behavior, user interaction, and accessibility.

Test Name	Description	File	Result
renders in the DOM	Mounts Test_Component and verifies the root element is present in the DOM.	Test_Component.cy.jsx	Pass
background-color is cornflowerblue	Verifies that the component renders with the expected default background color of cornflowerblue (rgb(100, 149, 237)).	Test_Component.cy.jsx	Pass
background-color changes on button click	Verifies that clicking the test button changes the component background color to pink (rgb(255, 0, 251)).	Test_Component.cy.jsx	Pass
automated accessibility tests with axe-core	Injects axe-core and runs a full accessibility audit on the mounted component, expecting no violations.	Test_Component.cy.jsx	Pass
add function adds numbers	Verifies that the add utility function returns the correct sum for given inputs ($7 + 3 = 10$).	test.spec.js	Pass
subtract function subtracts numbers	Verifies that the subtract utility function returns the correct difference for given inputs ($22 - 10$	test.spec.js	Pass

	= 12).		
--	--------	--	--

Defects

At this time, defects that were found were labeled as incidents. Many of what we consider defects were self-made defects that were identified and corrected during the implementation phase. Incidents were found that had the potential of turning into defects had they not been corrected. The two closed incidents related to authentication (AUTH-1.1-A and AUTH-2.1-A) are representative of this category. Incident SPEND-1.1-A remains open and is currently under investigation.

Summary

The testing procedure to date has been conducted using manual system testing and automated unit testing. Features tested include but are not limited to:

- Ability for a user to register an account with a unique username and email, subject to password complexity requirements

Test Cases: UserServiceTest — register_ (6 tests)*

- Ability for a registered and verified user to log in via username or email and password

Test Cases: UserServiceTest — login_ (4 tests)*

- Ability to verify an account via emailed token link, with expiration enforcement

Test Cases: UserServiceTest — verifyEmail_ (3 tests)*

- Ability for a user to reset their password using a time-limited emailed token

Test Cases: UserServiceTest — resetPassword_ (4 tests)*

- Ability for an authenticated user to view, add, and delete personal expense records via the Spending API

Test Cases: SpendingControllerTest — getExpenses_, addExpense_*, deleteExpense_* (10 tests)*

- Ability for the frontend Test_Component to render correctly, respond to user interaction, and pass automated accessibility checks

Test Cases: Test_Component.cy.jsx (4 tests), test.spec.js (2 tests)

Further system testing will be conducted as newly implemented features are completed. These features include full income tracking functionality, dashboard summary calculations, and transaction history filtering. To date, two high-severity incidents have been found and closed. One medium-severity incident remains open and is currently undergoing review.